Apunte Único: Algoritmos y Estructuras de Datos - Guía 2

Por alumnos de AED
Facultad de Ciencias Exactas y Naturales
UBA

Última actualización: 6 de abril de 2026

The Council Will Decide Your Fate:

(doble click en los ejercicios para saltar)

- Ejercicio 1
- Ejercicio 2
- Ejercicio 3
- Ejercicio 4
- Ejercicio 5
- Ejercicio 6
- Ejercicio 7
- Ejercicio 8
- Ejercicio 9
- Ejercicio 10
- Ejercicio 11
- Ejercicio 12
- Ejercicio 13
- Ejercicio 14
- Ejercicio 15
- Ejercicio 16
- Ejercicio 17
- Ejercicio 18
- Ejercicio 19
- Ejercicio 20
- Ejercicio 21
- Ejercicio 22
- Ejercicio 23
- Ejercicio 24
- Ejercicio 25

¡Recomendación para sacarle jugo al apunte!

Disclaimer: Recomendable leer lo siguiente, ayuda banda si no estas preparado para encarar la guía usando el repo.

Estudiar con resueltos puede ser un arma de doble filo. Si estás trabado, antes de saltar a la solución que hizo otra persona:

1. **No mires la solución ni bien te trabás.** Te condicionás pavlovianamente a no pensar. Necesitás darle tiempo al cerebro.
2. Intentá un ejercicio similar, pero más fácil. ¿No sale? Intentá uno aún más fácil.
3. Fijate si tenés un ejercicio similar hecho en clase.
4. Tomate 2 minutos para formular una pregunta real sobre lo que no entendés.

Eh loco, relajá un toque... ¡va a salir todo bien!

2.1. Predicados y Auxiliares

Ejercicio 1. Nombrar los siguientes predicados sobre enteros:

a) `pred ????(x: ℤ) { (∃ c: ℤ) (c >0 ∧ c * c = x) }`

b) `pred ????(x: ℤ) { (∀ n: ℤ) (1 <n <x → x mód n ≠ 0) }`

Resolución:

a) `esCuadradoPerfecto(x: ℤ)` (o `esCuadrado`):
Justificación: El predicado exige que exista un número entero c estrictamente positivo tal que, al multiplicarlo por sí mismo ($c \times c$), dé como resultado x. Esta es la definición matemática exacta de un cuadrado perfecto distinto de cero.

b) `esPrimo(x: ℤ)`:
Justificación: El predicado evalúa si para **todo** número entero n estrictamente mayor a 1 y menor a x, el resto de la división entera entre x y n (x mód n) es distinto de cero. Es decir, está verificando que x no tenga ningún divisor además del 1 y sí mismo, lo cual es la propiedad fundamental que define a un número primo.

Ejercicio 2. Escriba los siguientes predicados sobre números enteros en lenguaje de especificación:

a) `pred divide(x, y: ℤ)` que sea verdadero si y sólo si x divide a y (es decir, el resto es 0).

b) `pred sonCoprimos(x, y: ℤ)` que sea verdadero si y sólo si x e y son coprimos.

c) `pred mayorPrimoQueDivide(x: ℤ, y: ℤ)` que sea verdadero si y es el mayor primo que divide a x.

Resolución:

a) `pred divide(x: ℤ, y: ℤ)`
Podemos definirlo de dos maneras equivalentes. Usando la operación módulo (asegurándonos de que x no sea cero para no indefinir), o usando la definición formal con el cuantificador existencial. Cualquiera de las dos es válida:

- `pred divide(x: ℤ, y: ℤ) { x ≠ 0 ∧_L y mód x = 0 }`
- `pred divide(x: ℤ, y: ℤ) { (∃ k: ℤ) (x × k = y) }`

b) `pred sonCoprimos(x: ℤ, y: ℤ)`
Dos números son coprimos si el único divisor positivo que tienen en común es el 1. Acá empezamos a usar el predicado `divide` que armamos recién para modularizar:

```
pred sonCoprimos(x: ℤ, y: ℤ) {
                (∀ d: ℤ) (d >0 ∧ divide(d, x) ∧ divide(d, y) → d = 1) }
```

c) `pred mayorPrimoQueDivide(x: ℤ, y: ℤ)`
Desglosamos el problema en tres condiciones que deben cumplirse a la vez: 1. Que y sea primo (usamos el predicado del Ejercicio 1). 2. Que y divida a x. 3. Que cualquier otro

número p que también sea primo y divida a x, sea menor o igual a y (esto garantiza que y es el *mayor* de todos ellos).

```
pred mayorPrimoQueDivide(x: ℤ, y: ℤ) {
                        esPrimo(y) ∧ divide(y, x) ∧
                         (∀ p: ℤ) (esPrimo(p) ∧ divide(p, x) → p ≤ y) }
```

Ejercicio 3. Nombre los siguientes predicados auxiliares sobre secuencias de enteros:

a) `pred ????(s: seq⟨ℤ⟩) { (∀ i: ℤ) (0 ≤ i <|s| →L s[i] >0) }`

b) `pred ????(s: seq⟨ℤ⟩) { (∀ i: ℤ) (0 ≤ i <|s| → (∀ j: ℤ) (0 ≤ j <|s| ∧ i ≠ j →L s[i] ≠ s[j])) }`

Resolución:

a) `todosPositivos(s: seq⟨ℤ⟩)` (o `sonTodosPositivos`):
Justificación: El cuantificador universal recorre todos los índices válidos de la secuencia (desde 0 hasta $|s| - 1$). El operador de implicación "luego"($\rightarrow_L$) es clave acá: asegura que primero se cumpla que el índice está en rango antes de intentar acceder a $s[i]$, evitando que el programa explote. Finalmente, exige que todos los elementos en esas posiciones sean estrictamente mayores a cero.

b) `sinRepetidos(s: seq⟨ℤ⟩)` (o `elementosDistintos`):
Justificación: Plantea que para cualquier par de índices válidos i y j dentro de la secuencia, si los índices son distintos ($i \neq j$), entonces los valores guardados en esas posiciones también deben ser distintos ($s[i] \neq s[j]$). Esto garantiza matemáticamente que no haya ningún elemento duplicado en toda la secuencia.

Ejercicio 4. Escriba los siguientes predicados auxiliares sobre secuencias de enteros, aclarando los tipos de los parámetros que recibe:

a) `pred pertenece(...)` que determina si un elemento pertenece a una secuencia.

b) `pred esPrefijo(...)` que determina si una secuencia es prefijo de otra.

c) `pred estáOrdenada (...)` que determina si la secuencia está ordenada de menor a mayor.

d) `pred primosEnPosicionesPares(...)` que determina si todos los números primos están en posiciones pares.

e) `pred hayUnoPrimoQueDivideAlResto(...)` que determina si hay un elemento primo en la secuencia que divide a todos los otros elementos de la secuencia.

f) `pred enTresPartes (...)`, que determina si en la secuencia aparecen primero 0s, después 1s y por último 2s. ¿Cómo modificaría la expresión para que se admitan cero apariciones?

Resolución:

a) `pred pertenece(e: ℤ, s: seq⟨ℤ⟩)`
Basta con verificar si existe algún índice válido en la secuencia donde el elemento guardado coincida con e. Usamos $\wedge_L$ para evitar leer $s[i]$ si la secuencia estuviera vacía o el índice fuera de rango.

```
pred pertenece(e: ℤ, s: seq⟨ℤ⟩) { (∃ i: ℤ) (0 ≤ i <|s| ∧L s[i] = e) }
```

b) `pred esPrefijo(s1: seq⟨ℤ⟩, s2: seq⟨ℤ⟩)`
Para que $s1$ sea prefijo de $s2$, primero $s1$ debe ser de menor o igual longitud. Luego, todos los elementos de $s1$ deben coincidir exactamente con los primeros elementos de $s2$.

```
pred esPrefijo(s1: seq⟨ℤ⟩, s2: seq⟨ℤ⟩) {
            |s1| ≤ |s2| ∧ (∀ i: ℤ) (0 ≤ i <|s1| →L s1[i] = s2[i]) }
```

c) `pred estáOrdenada(s: seq⟨ℤ⟩)`
Comparamos cada elemento con el siguiente. Iteramos hasta $|s| - 1$ para que, al evaluar el último elemento, $s[i+1]$ no se caiga del arreglo.

```
pred estáOrdenada(s: seq⟨ℤ⟩) { (∀ i: ℤ) (0 ≤ i <|s| - 1 →L s[i] ≤ s[i+1]) }
```

d) `pred primosEnPosicionesPares(s: seq⟨ℤ⟩)`
Nota: Asumimos índices basados en 0 (el primer elemento está en la posición 0, que es par).
Decimos: "Para todo índice, SI el elemento en esa posición es primo, ENTONCES el índice debe ser par".

```
pred primosEnPosicionesPares(s: seq⟨ℤ⟩) {
            (∀ i: ℤ) (0 ≤ i <|s| ∧L esPrimo(s[i]) → i mód 2 = 0) }
```

e) `pred hayUnoPrimoQueDivideAlResto(s: seq⟨ℤ⟩)`
Reutilizamos el `esPrimo` y el `divide` (del Ejercicio 2).

```
pred hayUnoPrimoQueDivideAlResto(s: seq⟨ℤ⟩) {
            (∃ i: ℤ) (0 ≤ i <|s| ∧L esPrimo(s[i]) ∧
            (∀ j: ℤ) (0 ≤ j <|s| ∧ i ≠ j →L divide(s[i], s[j]))) }
```

f) `pred enTresPartes(s: seq⟨ℤ⟩)`
Para que haya estrictamente tres partes (al menos un 0, al menos un 1, y al menos un 2), deben existir dos índices de quiebre (los llamamos i y j). El primero marca dónde empiezan los 1s, y el segundo dónde empiezan los 2s.

```
pred enTresPartes(s: seq⟨ℤ⟩) {
            (∃ i, j: ℤ) (0 <i ∧ i <j ∧ j <|s| ∧
             (∀ k: ℤ)(0 ≤ k <i →L s[k] = 0) ∧
             (∀ k: ℤ)(i ≤ k <j →L s[k] = 1) ∧
                        (∀ k: ℤ)(j ≤ k <|s| →L s[k] = 2)) }
```

Modificación para admitir cero apariciones:
Si permitimos secuencias vacías o que falten bloques, los índices i y j pueden "pisarse.^entre sí o colapsar en los bordes. Cambiamos las desigualdades estrictas ($<$) por desigualdades amplias ($\leq$) en las condiciones de los índices de quiebre:

```
pred enTresPartesModificado(s: seq⟨ℤ⟩) {
                (∃ i, j: ℤ) (0 ≤ i ∧ i ≤ j ∧ j ≤ |s| ∧
                                ...(el resto de la fórmula se mantiene igual)...}
```

Ejercicio 5. Escribir una auxiliar que:

a) Sume 10 a un entero.

b) "Devuelva" el mayor elemento de una tupla de 3 enteros.

c) "Devuelva" el dígito menos significativo de un entero.

Resolución:

Recordemos que la sintaxis general para definir una función auxiliar en la materia es:
aux *nombre*(*parametros*) : *tipo* = *expresión*

a) Sume 10 a un entero
Simplemente tomamos un parámetro de tipo $\mathbb{Z}$ y le sumamos el valor constante.

$$\text{aux sumar10}(n : \mathbb{Z}) : \mathbb{Z} = n + 10$$

b) Mayor elemento de una tupla de 3 enteros
Sea t una tupla de tres elementos de tipo $\mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}$. Usualmente accedemos a los elementos de una tupla mediante subíndices (t_0, t_1, t_2). Podemos utilizar la función IfThenElse para armar la lógica condicional:

$$\text{aux maximoDeTres}(t : \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}) : \mathbb{Z} = \\ \text{IfThenElse}(t_0 \geq t_1 \wedge t_0 \geq t_2,\ t_0,\ \text{IfThenElse}(t_1 \geq t_2,\ t_1,\ t_2))$$

Nota: Otra forma 100 % válida (y más corta) si asumimos que ya existe una auxiliar básica `max(a,b)` *es escribirlo como* $max(t_0, max(t_1, t_2))$.

c) Dígito menos significativo de un entero
El dígito menos significativo de un número en base 10 se obtiene calculando el resto de su división entera por 10 (operación módulo). Para evitar resultados indeseados si el número llega a ser negativo, es una excelente práctica aplicarle el valor absoluto primero.

$$\text{aux digitoMenosSignificativo}(n : \mathbb{Z}) : \mathbb{Z} = |n| \text{ mód } 10$$

Ejercicio 6. Sea s una secuencia de elementos de tipo $\mathbb{Z}$. Escribir una auxiliar utilizando sumatoria y productoria que:

a) Cuente la cantidad de veces que aparece el elemento e de tipo $\mathbb{Z}$ en la secuencia s.

b) Sume los elementos en las posiciones pares de la secuencia s.

c) Sume los elementos mayores a 0 contenidos en la secuencia s.

d) Sume los inversos multiplicativos $(1/x)$ de los elementos contenidos en la secuencia s distintos a 0.

Resolución:

a) **Cuente la cantidad de veces que aparece el elemento e**
Usamos una sumatoria que recorre todos los índices de la secuencia. Por cada posición, evaluamos si el elemento es igual a e. Si es igual sumamos 1, si no, sumamos 0.

$$\text{aux apariciones}(e : \mathbb{Z}, s : \text{seq}\langle\mathbb{Z}\rangle) : \mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] = e, 1, 0)$$

b) **Sume los elementos en las posiciones pares**
Recorremos todos los índices y usamos la operación módulo para verificar si el índice i es par. Si lo es, sumamos el valor de la secuencia en esa posición ($s[i]$), caso contrario sumamos 0.

$$\text{aux sumaPosicionesPares}(s : \text{seq}\langle\mathbb{Z}\rangle) : \mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(i \text{ mód } 2 = 0, s[i], 0)$$

c) **Sume los elementos mayores a 0**
Similar a los anteriores, pero la condición del `IfThenElse` filtra por los valores estrictamente positivos de la secuencia.

$$\text{aux sumaPositivos}(s : \text{seq}\langle\mathbb{Z}\rangle) : \mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] > 0, s[i], 0)$$

d) **Sume los inversos multiplicativos de los elementos distintos a 0**
Acá debemos tener cuidado de no dividir por cero. Al agregar la condición $s[i] \neq 0$, garantizamos que la cuenta sea segura. Ojo con un detalle importante: como el inverso multiplicativo es una división $(1/x)$, nuestra sumatoria ya no va a devolver un entero, ¡sino un número Real ($\mathbb{R}$)!

$$\text{aux sumaInversos}(s : \text{seq}\langle\mathbb{Z}\rangle) : \mathbb{R} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] \neq 0, \frac{1}{s[i]}, 0)$$

2.2. Análisis de especificación

Ejercicio 7. Las siguientes especificaciones no son correctas. Indicar por qué y corregirlas para que describan correctamente el problema.

a) `proc progresionGeometricaFactor2()` Indica si la secuencia `l` representa una progresión geométrica factor 2. Es decir, si cada elemento de la secuencia es el doble del elemento anterior.

```
proc progresionGeometricaFactor2 (in l: seq⟨ℤ⟩): bool {
   requiere { True }
   asegura { res = True ↔ (∀ i: ℤ) (0 ≤ i <|l| →L l[i] = 2 * l[i - 1]) }
}
```

b) `proc mínimo()` Devuelve en `res` el menor elemento de `l`.

```
proc mínimo (in l: seq⟨ℤ⟩): ℤ {
   requiere { True }
   asegura { (∀ y: ℤ)((y ∈ l ∧ y ≠ x) → y >res) }
}
```

Resolución:

a) **Errores en `progresionGeometricaFactor2`:**

- **Índice fuera de rango:** El cuantificador universal evalúa desde $i = 0$. Cuando $i = 0$, la expresión intenta acceder a `l[i - 1]`, es decir, `l[-1]`, lo cual indefine la expresión (rompe el programa).

Corrección: Cambiamos el rango para que arranque a evaluar desde $i = 1$.

```
proc progresionGeometricaFactor2 (in l: seq⟨ℤ⟩): bool {
   requiere { True }
   asegura { res = True ↔ (∀ i: ℤ) (0 <i <|l| →L l[i] = 2 * l[i - 1]) }
}
```

Nota: Si la secuencia tiene longitud 0 o 1, el antecedente $0 < i < |l|$ es falso, por lo que el $\forall$ da Verdadero por vacuidad, lo cual es correcto matemático (una secuencia vacía o de un elemento es trivialmente una progresión).

b) **Errores en `mínimo`:**

- **Precondición insuficiente:** No se puede obtener el mínimo de una secuencia vacía. El `requiere` debe exigir $|l| > 0$.
- **Variable no declarada:** Se utiliza una variable x que no existe ni en los parámetros ni está cuantificada.
- **No asegura pertenencia:** Nunca se exige que el resultado (`res`) sea un elemento que efectivamente pertenezca a la secuencia `l`.
- **Manejo de duplicados:** Al usar la desigualdad estricta ($y > res$), falla si el mínimo aparece repetido en la secuencia.

Corrección: Exigimos que la secuencia tenga elementos, que `res` pertenezca a la secuencia, y que sea menor o igual a todos los demás.

```
proc mínimo (in l: seq⟨ℤ⟩): ℤ {
   requiere { |l| >0 }
   asegura { pertenece(res, l) ∧ (∀ i: ℤ)(0 ≤ i <|l| →L res ≤ l[i]) }
}
```

Ejercicio 8. Para los siguientes problemas, dar todas las soluciones posibles a las entradas dadas:

a)
```
proc indiceDelMaximo (in l: seq⟨ℝ⟩): ℤ {
   requiere { |l| >0 }
   asegura { 0 ≤ res <|l| ∧L (∀ i: ℤ) (0 ≤ i <|l| →L l[i] ≤ l[res]) }
}
```

I) $l = \langle 1; 2; 3; 4 \rangle$

II) $l = \langle 15{,}5; -18; 4{,}215; 15{,}5; -1 \rangle$

III) $l = \langle 0; 0; 0; 0; 0; 0 \rangle$

b)
```
proc indiceDelPrimerMaximo (in l: seq⟨ℝ⟩): ℤ {
   requiere { |l| >0 }
   asegura { 0 ≤ res <|l| ∧L (∀ i: ℤ) (0 ≤ i <|l| →L (l[i] <l[res] ∨ (l[i]
= l[res] ∧ i ≥ res))) }
}
```

I) $l = \langle 1; 2; 3; 4 \rangle$

II) $l = \langle 15{,}5; -18; 4{,}215; 15{,}5; -1 \rangle$

III) $l = \langle 0; 0; 0; 0; 0; 0 \rangle$

c) ¿Para qué valores de entrada `indiceDelPrimerMaximo` y `indiceDelMaximo` tienen necesariamente la misma salida?

Resolución:

a) En este procedimiento, la postcondición simplemente exige que el elemento en la posición `res` sea mayor o igual que todos los demás elementos. Es decir, `res` puede ser el índice de **cualquier** aparición del valor máximo.

I) **Salidas posibles:** `res = 3` (el máximo es 4).

II) **Salidas posibles:** `res = 0` ó `res = 3` (el máximo 15.5 aparece en ambos índices).

III) **Salidas posibles:** `res` $\in \{0, 1, 2, 3, 4, 5\}$ (todos los elementos son el máximo, 0, así que cualquier índice es válido).

b) Aquí la postcondición agrega una restricción clave: si hay otro elemento igual al máximo (`l[i] = l[res]`), su índice `i` debe ser obligatoriamente mayor o igual a `res`. Esto fuerza a que `res` sea estrictamente el **primer** índice donde aparece el máximo (el que está más a la izquierda).

I) **Salidas posibles:** `res = 3`.

ii) **Salidas posibles:** `res = 0` (es la primera aparición del 15.5).

iii) **Salidas posibles:** `res = 0` (es la primera aparición del 0).

c) Tienen necesariamente la misma salida **sólo cuando el elemento máximo aparece exactamente una sola vez** en la secuencia.
Si el máximo aparece más de una vez (como en los casos ii y iii), `indiceDelMaximo` se vuelve no determinístico y permite devolver varias opciones, mientras que `indiceDelPrimerMaximo` está obligado a devolver únicamente la primera aparición.

Ejercicio 9. Sea $f : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ definida como:

$$f(a,b) = \begin{cases} 2 \times b & \textit{si } a < 0 \\ b - 1 & \textit{en otro caso} \end{cases}$$

Indicar cuáles de las siguientes especificaciones son correctas para el problema de calcular $f(a,b)$. Para aquellas que no lo son, indicar por qué.

a) `asegura { (a <0 ∧ res = 2 × b) ∧ (a ≥ 0 ∧ res = b - 1) }`

b) `asegura { (a <0 ∧ res = 2 × b) ∨ (a ≥ 0 ∧ res = b - 1) }`

c) `asegura { (a <0 → res = 2 × b) ∨ (a ≥ 0 → res = b - 1) }`

d) `asegura { res = IfThenElse(a <0, 2 × b, b - 1) }`

(Nota: Omitimos la cabecera del proc y el requiere {True} por brevedad, evaluamos solo el asegura).

Resolución:

a) Incorrecta.
Motivo: Utiliza una conjunción ($\wedge$) para unir los dos casos. Como es imposible que $a < 0$ y $a \geq 0$ sean verdaderos al mismo tiempo para un mismo número, la expresión completa siempre evaluará a `False`. Una postcondición que siempre es falsa significa que es imposible escribir un programa que la cumpla.

b) Correcta.
Motivo: Utiliza una disyunción ($\vee$). Como los dominios de a ($a < 0$ y $a \geq 0$) son mutuamente excluyentes y cubren todos los casos posibles, siempre una de las dos mitades del $\vee$ será falsa por culpa de a, y la otra mitad será la encargada de exigir que `res` tenga el valor correcto. Es la forma clásica de escribir un 'if-else' con lógica proposicional.

c) Incorrecta.
Motivo: Es un error lógico grave ("la trampa de la implicación con el OR"). Supongamos que $a = 5$. La primera parte es ($5 < 0 \to res = 2 \times b$). Como el antecedente es Falso, la implicación completa es **Verdadera**. Como están unidas por un $\vee$ (OR), basta con que esa primera mitad sea Verdadera para que **toda** la postcondición dé Verdadero, sin importar qué valor tome `res`. Es decir, si $a = 5$, el algoritmo podría devolver cualquier verdura y la postcondición seguiría dando True.

d) **Correcta.**
Motivo: Utiliza la función auxiliar predefinida `IfThenElse`, la cual mapea exactamente con la definición de una función partida matemática. Es la forma más legible y directa de especificar este problema.

Ejercicio 10. Considerar la siguiente especificación, junto con un algoritmo que dado x devuelve x^2:

```
proc unoMasGrande (in x: R): R {
    requiere { True }
    asegura { res > x }
}
```

a) ¿Qué devuelve el algoritmo si recibe $x = 3$? ¿El resultado hace verdadera la postcondición de `unoMasGrande`?

b) ¿Qué sucede para las entradas $x = 0{,}5$, $x = 1$, $x = -0{,}2$ y $x = -7$?

c) Teniendo en cuenta lo respondido en los puntos anteriores, escribir una precondición para `unoMasGrande`, de manera tal que el algoritmo cumpla con la especificación.

Resolución:

a) **Para $x = 3$:**
El algoritmo devuelve $x^2 = 3^2 = 9$.
Evaluamos la postcondición (`res >x`): $9 > 3$. Como esto es Verdadero, el resultado **sí hace verdadera la postcondición**.

b) **Evaluación de otras entradas:**

- Si $x = 0{,}5 \to \text{res} = 0{,}5^2 = 0{,}25$. Postcondición ($0{,}25 > 0{,}5$): **Falsa**.
- Si $x = 1 \to \text{res} = 1^2 = 1$. Postcondición ($1 > 1$): **Falsa**.
- Si $x = -0{,}2 \to \text{res} = (-0{,}2)^2 = 0{,}04$. Postcondición ($0{,}04 > -0{,}2$): **Verdadera**.
- Si $x = -7 \to \text{res} = (-7)^2 = 49$. Postcondición ($49 > -7$): **Verdadera**.

c) **Precondición corregida:**
Para que el algoritmo siempre garantice que la postcondición sea verdadera, necesitamos restringir el dominio de entrada (el `requiere`) para que solo acepte valores de x donde la inecuación $x^2 > x$ se cumpla matemáticamente.

Resolvemos la inecuación:

$$x^2 > x$$

$$x(x-1) > 0$$

Esta parábola es positiva (mayor a cero) cuando x está por fuera del intervalo de sus raíces (que son 0 y 1). Es decir, la condición se cumple estrictamente si $x < 0$ o si $x > 1$.
Por lo tanto, la precondición fuerte que hace que el algoritmo cumpla la especificación es:

`requiere { x <0 ∨ x >1 }`

2.3. Relación de Fuerza

Ejercicio 11. Sean x y res variables de tipo $\mathbb{R}$. Considerar los siguientes predicados:

- $P_1 : \{x \leq 0\}$, $P_2 : \{x \leq 10\}$, $P_3 : \{x \leq -10\}$
- $Q_1 : \{res \geq x^2\}$, $Q_2 : \{res \geq 0\}$, $Q_3 : \{res = x^2\}$

a) Indicar la relación de fuerza entre P_1, P_2 y P_3.

b) Indicar la relación de fuerza entre Q_1, Q_2 y Q_3.

c) Escribir 2 programas que cumplan con la siguiente especificación:

```
proc hagoAlgo (in x: ℝ): ℝ {
   requiere { x ≤ 0 }
   asegura { res ≥ x² }
}
```

d) Sea A un algoritmo que cumple con la especificación del ítem anterior. Decidir si necesariamente cumple las siguientes especificaciones:

I) requiere $\{x \leq -10\}$, asegura $\{res \geq x^2\}$

II) requiere $\{x \leq 10\}$, asegura $\{res \geq x^2\}$

III) requiere $\{x \leq 0\}$, asegura $\{res \geq 0\}$

IV) requiere $\{x \leq 0\}$, asegura $\{res = x^2\}$

V) requiere $\{x \leq -10\}$, asegura $\{res \geq 0\}$

VI) requiere $\{x \leq 10\}$, asegura $\{res = x^2\}$

e) ¿Qué conclusión pueden sacar? ¿Qué debe cumplirse con respecto a las precondiciones y postcondiciones para que sea seguro reemplazar la especificación?

Resolución:

a) Relación de fuerza entre P_1, P_2 y P_3:
Recordemos que A es más fuerte que B si $A \to B$.
Si un número es menor a -10, seguro es menor a 0, y seguro es menor a 10. Por lo tanto: $P_3 \to P_1 \to P_2$.
P_3 es el más fuerte, P_1 es intermedio, y P_2 es el más débil.

b) Relación de fuerza entre Q_1, Q_2 y Q_3:
Sabemos por matemática que todo número real al cuadrado es mayor o igual a cero ($x^2 \geq 0$).
Si $res = x^2$, entonces seguro $res \geq x^2$. Y si $res \geq x^2$, entonces seguro $res \geq 0$. Por lo tanto: $Q_3 \to Q_1 \to Q_2$.
Q_3 es el más fuerte, Q_1 es intermedio, y Q_2 es el más débil.

c) Dos programas que cumplan la especificación:

- **Programa 1:** `res = x * x` (Cumple con el límite exacto de la postcondición).
- **Programa 2:** `res = (x * x) + 1` (Como $x^2 + 1 \geq x^2$, este programa también hace verdadera la postcondición, demostrando el no-determinismo de la especificación).

d) Análisis del Algoritmo A:

I) **Sí cumple.** Al pedir $x \leq -10$, le estamos dando al algoritmo datos aún más restringidos de los que necesita ($x \leq 0$). El algoritmo funcionará perfecto.

II) **No cumple.** Si le pasamos $x = 5$ (permitido por $x \leq 10$), rompemos la precondición original del algoritmo ($x \leq 0$) y su comportamiento se indefine (puede fallar).

III) **Sí cumple.** El algoritmo garantiza $res \geq x^2$. Como eso implica matemáticamente que $res \geq 0$, la nueva postcondición se satisface "de sobra".

IV) **No cumple.** El algoritmo A podría ser el "Programa 2"del inciso anterior. Devolvería $x^2 + 1$, lo cual hace falsa la nueva postcondición estricta $res = x^2$.

V) **Sí cumple.** Combina una entrada más segura (i) y una salida menos exigente (iii). Es el escenario más seguro posible.

VI) **No cumple.** Falla por partida doble: permite entradas inválidas para A y exige salidas exactas que A no garantiza.

e) Conclusión:
Para que sea seguro utilizar un algoritmo en reemplazo de otro (o decir que cumple una nueva especificación), se deben cumplir simultáneamente dos reglas:

- **Precondición:** La nueva precondición debe ser **igual o más fuerte** que la original ($Pre_{nueva} \rightarrow Pre_{original}$). Esto asegura que el entorno filtrará los datos y el algoritmo nunca recibirá valores que no sepa manejar.
- **Postcondición:** La nueva postcondición debe ser **igual o más débil** que la original ($Post_{original} \rightarrow Post_{nueva}$). Esto asegura que las garantías que ya daba el algoritmo son más que suficientes para satisfacer lo que ahora se pide.

Ejercicio 12. Considerar las siguientes dos especificaciones, junto con un algoritmo a que satisface la especificación de `p2`.

```
proc p1(in x: ℝ, in n: ℤ): ℤ {
  requiere { x ≠ 0 }
  asegura { x^n - 1 <res ≤ x^n }
}

proc p2(in x: ℝ, in n: ℤ): ℤ {
  requiere { n ≤ 0 → x ≠ 0 }
  asegura { res = ⌊x^n⌋ }
}
```

a) Dados valores de x y n que hacen verdadera la precondición de `p1`, demostrar que hacen también verdadera la precondición de `p2`.

b) Ahora, dados estos valores de x y n, supongamos que se ejecuta a: llegamos a un valor de `res` que hace verdadera la postcondición de `p2`. ¿Será también verdadera la postcondición de `p1` con este valor de `res`?

c) ¿Podemos concluir que a satisface la especificación de `p1`?

Resolución:

a) Sea $Pre_{p1} \equiv x \neq 0$ y $Pre_{p2} \equiv (n \leq 0 \rightarrow x \neq 0)$.
Debemos demostrar que $Pre_{p1} \rightarrow Pre_{p2}$.
Asumimos que Pre_{p1} es verdadera, es decir, sabemos como hecho que $x \neq 0$.
Analizamos Pre_{p2}: es una implicación lógica. Como el consecuente de la implicación $(x \neq 0)$ es verdadero por nuestra asunción, toda la implicación $(n \leq 0 \rightarrow x \neq 0)$ resulta verdadera, sin importar el valor que tome n.
Por lo tanto, la precondición de `p2` también es verdadera.

b) Sea $Post_{p2} \equiv res = \lfloor x^n \rfloor$ y $Post_{p1} \equiv x^n - 1 < res \leq x^n$.
Debemos analizar si $Post_{p2} \rightarrow Post_{p1}$.
Asumimos que $res = \lfloor x^n \rfloor$. Por la definición matemática de la función piso o parte entera ($\lfloor y \rfloor$), sabemos que devuelve el mayor número entero menor o igual a y. Por propiedad fundamental, se cumple para cualquier valor y que:

$$y - 1 < \lfloor y \rfloor \leq y$$

Reemplazando y por x^n, obtenemos:

$$x^n - 1 < \lfloor x^n \rfloor \leq x^n$$

Como sabemos que $res = \lfloor x^n \rfloor$, reemplazamos en la inecuación:

$$x^n - 1 < res \leq x^n$$

¡Que es exactamente $Post_{p1}$! Por lo tanto, **sí, será verdadera**.

c) Sí, podemos concluir que a satisface la especificación de `p1`.
Para afirmar esto de forma rigurosa, se deben cumplir las dos condiciones de reemplazo seguro que dedujimos en el Ejercicio 11:

- $Pre_{p1} \rightarrow Pre_{p2}$ (Demostrado en el inciso a): Garantiza que `p1` filtra las entradas de forma más exigente, haciéndolas seguras para ejecutar el algoritmo a (que asume algo más laxo en Pre_{p2}).
- $Post_{p2} \rightarrow Post_{p1}$ (Demostrado en el inciso b): Garantiza que las salidas de a (que asegura $Post_{p2}$) son suficientes para cumplir lo que se le exige a `p1`.

2.3. Relación de Fuerza

Ejercicio 13. Especificar los siguientes problemas:

a) Dado un entero, decidir si es par.

b) Dado un entero n y otro m, decidir si n es un múltiplo de m.

c) Dado un entero, listar todos sus divisores positivos (sin duplicados).

d) Dado un entero positivo, obtener su descomposición en factores primos. Devolver una secuencia de tuplas (p, e), donde p es un factor primo y e es su exponente, ordenada en forma creciente con respecto a p.

Resolución:

a) **Decidir si es par**
Simplemente verificamos que el resto de la división por 2 sea cero.

```
proc esPar (in x: ℤ): bool {
  requiere { True }
  asegura { res = True ↔ x mód 2 = 0 }
}
```

b) **Decidir si n es múltiplo de m**
*Nota: Si usamos el operador módulo (*mód*), debemos exigir en la precondición que $m \neq 0$ para evitar una división por cero.*

```
proc esMultiplo (in n: ℤ, in m: ℤ): bool {
  requiere { m ≠ 0 }
  asegura { res = True ↔ n mód m = 0 }
}
```

c) **Listar todos sus divisores positivos (sin duplicados)**
Utilizamos la función auxiliar `sinRepetidos` (del Ejercicio 3) y la función `divide` (del Ejercicio 2). Pedimos que $x \neq 0$ para que la cantidad de divisores sea finita.

```
proc divisoresPositivos (in x: ℤ): seq⟨ℤ⟩ {
  requiere { x ≠ 0 }
  asegura { sinRepetidos(res) ∧
      (∀ d: ℤ)(pertenece(d, res) ↔ (d >0 ∧ divide(d, x))) }
}
```

d) **Descomposición en factores primos**
Representamos las tuplas como elementos de $\mathbb{Z} \times \mathbb{Z}$, donde la primera componente (t_0) es el primo p y la segunda (t_1) es el exponente e. Por el Teorema Fundamental de la Aritmética, si garantizamos que todas las bases en la secuencia son primas, que todos los exponentes son positivos (mayores a cero), y que el producto total da x, entonces es la factorización correcta.

```
proc factoresPrimos (in x: ℤ): seq⟨ℤ × ℤ⟩ {
   requiere { x >0 }
   asegura { primosOrdenados(res) ∧ productoFactores(res) = x }
}

aux primosOrdenados(s: seq⟨ℤ × ℤ⟩): bool =
    (∀ i: ℤ)(0 ≤ i <|s| →_L (esPrimo(s[i]_0) ∧ s[i]_1 >0)) ∧
    (∀ i: ℤ)(0 ≤ i <|s| - 1 →_L s[i]_0 <s[i+1]_0)

aux productoFactores(s: seq⟨ℤ × ℤ⟩): ℤ = ∏_{i=0}^{|s|-1} s[i]_0^{s[i]_1}
```

Ejercicio 14. Especificar los siguientes problemas sobre secuencias:

a) Dadas dos secuencias s y t, decidir si s está incluida en t (todos los elementos de s aparecen en t en igual o mayor cantidad).

b) Dadas dos secuencias s y t, devolver su intersección (cantidad mínima de apariciones si hay repetidos).

c) Dada una secuencia de números enteros, devolver aquel que divida a más elementos de la secuencia.

d) Dada una secuencia de secuencias de enteros l, devolver una secuencia de l que contenga el máximo valor.

e) Dada una secuencia l con todos sus elementos distintos, devolver la secuencia de partes (todas las subsecuencias manteniendo el orden).

Resolución:

Nota: Para mantener las especificaciones legibles, asumiremos la existencia de auxiliares básicas vistas en ejercicios anteriores, como `apariciones(e, s)`, `pertenece(e, s)` *y* `divide(a, b)`.

a) Decidir si s está incluida en t

Basta con pedir que para cualquier número entero, su cantidad de apariciones en s sea menor o igual a sus apariciones en t. Si un elemento no está en s, sus apariciones son 0, lo cual siempre es $\leq$ a lo que haya en t.

```
proc incluida (in s: seq⟨ℤ⟩, in t: seq⟨ℤ⟩): bool {
   requiere { True }
   asegura { res = True ↔ (∀ e: ℤ)(apariciones(e, s) ≤ apariciones(e, t)) }
}
```

b) Intersección de dos secuencias

Exigimos que las apariciones de cada elemento en la secuencia resultado (`res`) sea exactamente el mínimo entre sus apariciones en s y en t.

```
proc interseccion (in s: seq⟨ℤ⟩, in t: seq⟨ℤ⟩): seq⟨ℤ⟩ {
   requiere { True }
   asegura { (∀ e: ℤ)(apariciones(e, res) = min(apariciones(e, s), apariciones(e,
t))) }
}
aux min(a: ℤ, b: ℤ): ℤ = IfThenElse(a <b, a, b)
```

c) Elemento que divide a más elementos

Necesitamos que la secuencia tenga elementos para poder devolver uno. Luego, el `res` debe pertenecer a la secuencia y tener una cantidad de "divisibles" mayor o igual a la de cualquier otro elemento que también pertenezca a la secuencia.

```
proc divideAMas (in s: seq⟨ℤ⟩): ℤ {
   requiere { |s| >0 }
   asegura { pertenece(res, s) ∧
       (∀ e: ℤ)(pertenece(e, s) → divisibles(res, s) ≥ divisibles(e, s)) }
}
aux divisibles(d: ℤ, s: seq⟨ℤ⟩): ℤ = ∑_{i=0}^{|s|−1} IfThenElse(divide(d, s[i]), 1, 0)
```

d) Secuencia que contenga el máximo valor absoluto

Para que exista un máximo, la secuencia de secuencias debe tener al menos una secuencia adentro, y esa secuencia no puede ser vacía.

```
proc seqConMaximo (in l: seq⟨seq⟨ℤ⟩⟩): seq⟨ℤ⟩ {
   requiere { |l| >0 ∧ (∀ i: ℤ)(0 ≤ i <|l| →_L |l[i]| >0) }
   asegura { perteneceSeq(res, l) ∧
       (∃ e: ℤ)(pertenece(e, res) ∧ esMaximoAbsoluto(e, l)) }
}
aux esMaximoAbsoluto(e: ℤ, l: seq⟨seq⟨ℤ⟩⟩): bool =
   (∀ i: ℤ)(0 ≤ i <|l| →_L (∀ j: ℤ)(0 ≤ j <|l[i]| →_L e ≥ l[i][j]))
```

e) Secuencia de partes (subconjuntos manteniendo orden)

La forma más elegante de definir que s es una subsecuencia de l manteniendo el orden, es postular que existe una 'secuencia de índices' estrictamente creciente que mapea los elementos de s con sus posiciones originales en l.

```
proc partes (in l: seq⟨ℤ⟩): seq⟨seq⟨ℤ⟩⟩ {
   requiere { sinRepetidos(l) }
   asegura { sinRepetidosSeq(res) ∧
       (∀ s: seq⟨ℤ⟩)(perteneceSeq(s, res) ↔ esSubsecuencia(s, l)) }
}
aux esSubsecuencia(s: seq⟨ℤ⟩, l: seq⟨ℤ⟩): bool = (∃ idx: seq⟨ℤ⟩) (
    |idx| = |s| ∧ estrictamenteCreciente(idx) ∧
    (∀ i: ℤ)(0 ≤ i <|idx| →_L (0 ≤ idx[i] <|l| ∧_L l[idx[i]] = s[i])) )
```

Ejercicio 15. Dados dos enteros a y b, se necesita calcular su suma y retornarla en un entero c. ¿Cuáles de las siguientes especificaciones son correctas para este problema? Para las que no lo son, indicar por qué.

a)
```
proc sumar (inout a, b, c: ℤ) {
   requiere { True }
   asegura { a + b = c }
}
```

b)
```
proc sumar (in a, b: ℤ, inout c: ℤ) {
   requiere { True }
   asegura { c = a + b }
}
```

c) `proc sumar(inout a, b: ℤ, inout c: ℤ) {`
   `requiere { a = A₀ ∧ b = B₀ }`
   `asegura { a = A₀ ∧ b = B₀ ∧ c = a + b }`
`}`

Resolución:

a) **Incorrecta.**
Motivo: Al declarar a y b como parámetros `inout`, el procedimiento tiene permiso para modificarlos. En la cláusula `asegura`, las variables representan sus valores **finales** (después de ejecutar el código). Esta especificación solo exige que, al final, el valor de c sea la suma de los valores resultantes de a y b. Un programador podría escribir `a = 0; b = 0; c = 0;` y cumpliría perfectamente la postcondición ($0 + 0 = 0$), pero perdiendo por completo los números originales que queríamos sumar.

b) **Correcta.**
Motivo: Al usar el modo `in` para a y b, el lenguaje de especificación garantiza que el procedimiento no puede modificarlos (son variables de solo lectura). Por lo tanto, en la cláusula `asegura`, a y b valen obligatoriamente lo mismo que al principio de la ejecución. Exigir $c = a + b$ es totalmente seguro y hace exactamente lo que pide el enunciado.

c) **Correcta (aunque verbosa).**
Motivo: A pesar de que a y b están declarados como `inout` (lo cual permite modificarlos), la especificación se .ªta de manos.ª sí misma utilizando las variables lógicas en mayúscula (A_0 y B_0). En el `requiere` "saca una foto"de los valores iniciales, y en el `asegura` obliga a que los valores finales de a y b sean idénticos a esa foto ($a = A_0 \wedge b = B_0$). Como garantizamos que no cambiaron, afirmar luego que $c = a + b$ es correcto. De todas formas, la opción **b** es la práctica ideal en AED.